

Complément de conception

Intégration de l'API BDS – Système RAG pour hcp.ma

Structure d'accueil : Haut-Commissariat au Plan (HCP), Direction des Systèmes
d'Information Statistiques

Binôme : Ayman El Baida et Saad Mesbah

Période du stage : 1er juillet – fin août 2026

*Document complémentaire au dossier de conception ([conception_uml_v3.pdf](#)) – suite à
l'ADR 0004, 15 juillet 2026*

Table des matières

1 Introduction

Ce document ne remplace pas le dossier de conception initial (`conception_uml_v3.pdf`) mais le complète : il documente les écarts de conception introduits par la découverte de l'API de la BDS (`bds.hcp.ma`), décidée dans l'ADR 0004 (`docs/adr/0004-api-bds-pour-les-indicateurs.md`). Deux questions de conception ont été tranchées : comment garder chaque indicateur traçable jusqu'à une source alors qu'il ne provient plus d'un document scrapé, et comment concilier fraîcheur des données et temps de réponse. Ce document formalise les deux réponses retenues et leur impact sur le modèle logique de données et sur le module `LookupStructure`.

2 Delta au modèle logique de données

Le MCD/MLD original (figures 2 et 3 de `conception_uml_v3.pdf`) reste structurellement valide : aucune entité, aucune association n'est ajoutée ou supprimée. Deux colonnes suffisent à accueillir la nouvelle source.

Table	Colonne	Changement
document	type	Le <code>CHECK</code> passe de <code>('html','pdf','xlsx')</code> à <code>('html','pdf','xlsx','api')</code> . Un document de type <code>api</code> est une fiche synthétique créée automatiquement pour représenter un indicateur BDS : <code>url</code> pointe vers <code>bds.hcp.ma/main/indicators/{code}</code> , <code>titre</code> reprend le libellé de l'indicateur. Il n'existe pas de HTML/PDF réel derrière – c'est un objet de traçabilité, pas un fichier collecté.
indicateur	code_bds	Nouvelle colonne, nullable (<code>NULL</code> pour un indicateur extrait d'un PDF/XLSX). Contient le code BDS (ex. <code>I3181</code>) quand la ligne vient de l'API. Indexée : c'est la clé utilisée par le cache pour savoir, avant tout appel réseau, si un indicateur a déjà été récupéré.

Pourquoi une fiche synthétique plutôt qu'une exception au schéma

La contrainte `indicateur.id_document NOT NULL` existait pour garantir l'exigence NF3 (traçabilité systématique de chaque réponse chiffrée jusqu'à sa source). Deux options s'offraient : assouplir la contrainte (rendre `id_document` nullable pour les indicateurs API), ou créer un document synthétique qui joue le rôle de source pour ces indicateurs. La première option crée une exception silencieuse : un indicateur sans document est un indicateur non traçable, ce qui contredit directement NF3. La seconde option (retenue) garde le schéma intact et rend la traçabilité explicite : même un indicateur venu d'un appel API pointe vers quelque chose de citable, la fiche officielle de l'indicateur sur `bds.hcp.ma`.

3 Diagramme d'activité – traitement d'une question chiffrée (avec repli API)

Ce diagramme remplace, pour les questions chiffrées, la partie `LookupStructure` du diagramme de séquence de la figure 5 (`conception_uml_v3.pdf`). Il décrit la stratégie *cache-aside* : vérifier le cache local avant tout appel réseau, et n'appeler l'API qu'en dernier recours.

Analyse

Le chemin le plus court (cache trouvé dès la première étape) reste très rapide : c'est le cas majoritaire attendu, grâce au pré-remplissage nocturne (figure B). Les deux points de sortie « indicateur non disponible » matérialisent directement le besoin fonctionnel F4 : le système ne doit jamais inventer

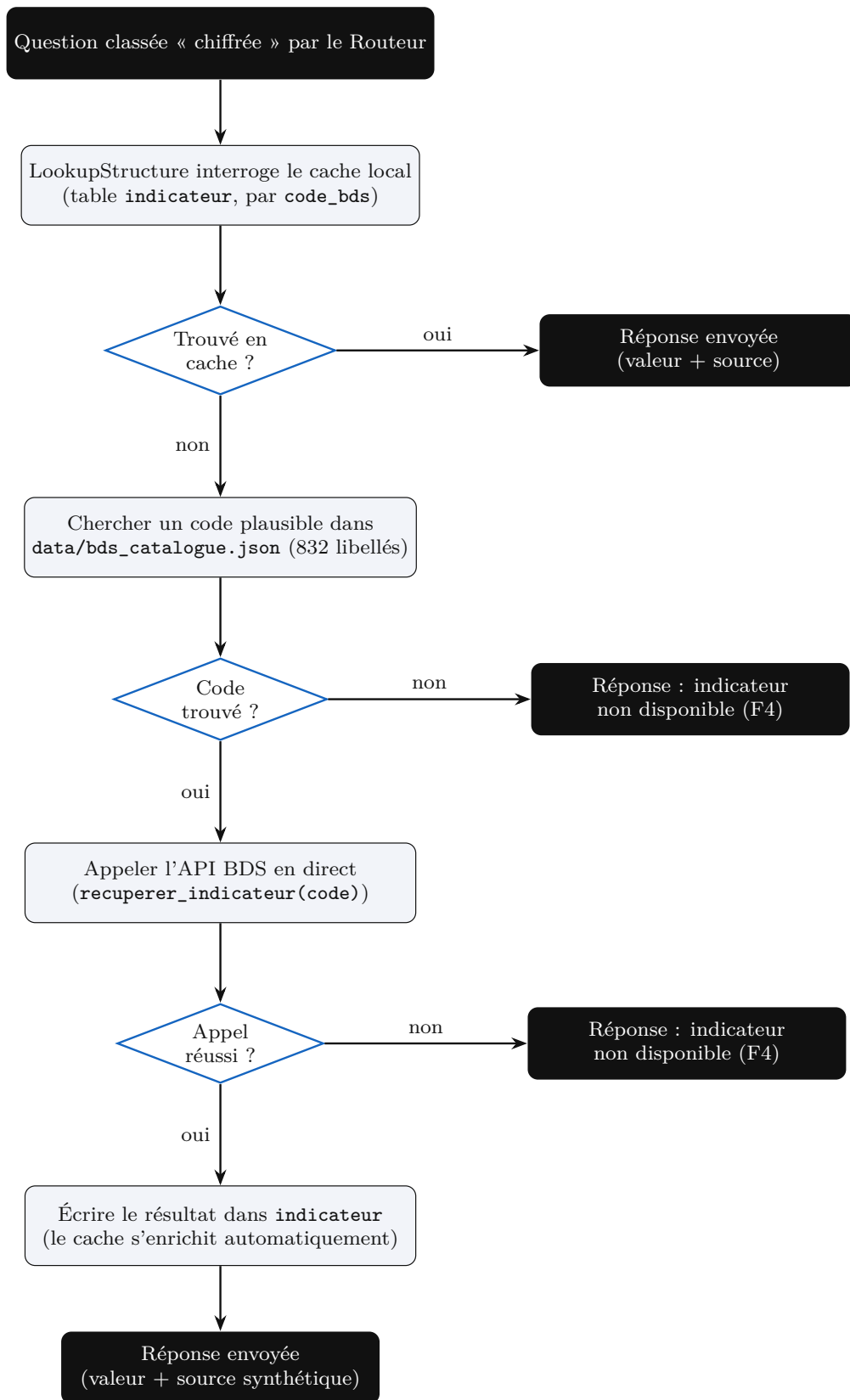


Figure A – Lookup hybride cache-aside pour une question chiffrée

une valeur chiffrée, il doit reconnaître explicitement les limites de sa couverture. La dernière étape (écriture dans **indicateur** après un appel réussi) est ce qui rend le cache auto-expansif : un indicateur demandé une fois en direct devient disponible en cache pour toutes les questions suivantes portant sur le même code.

4 Diagramme – pré-remplissage nocturne

En complément du chemin « à la demande » ci-dessus, un traitement par lot maintient le cache à jour pour les indicateurs les plus consultés, sans attendre qu'un utilisateur les demande.

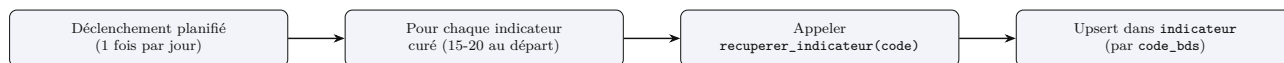


Figure B – Pré-remplissage nocturne des indicateurs curés

Analyse

Cette tâche planifiée est indépendante du chemin utilisateur : elle ne bloque jamais une réponse, elle prépare le terrain. L'« upsert » (mise à jour si le `code_bds` existe déjà, insertion sinon) garantit que les valeurs restent alignées sur les dernières publications de la BDS sans dupliquer de lignes à chaque exécution.

5 Impact sur les modules existants

Module	Impact
LookupStructure	Passe d'une simple requête SQL à la cascade décrite en figure A. Doit désormais savoir interroger <code>data/bds_catalogue.json</code> et appeler <code>src/bds_client.py</code> en repli.
ConstructeurIndicateurs	Reste responsable des indicateurs extraits de PDF/XLSX (<code>code_bds</code> IS NULL), mais n'est plus la seule voie d'alimentation de la table <code>indicateur</code> : le pré-remplissage nocturne et le repli en direct l'alimentent aussi, via <code>code_bds</code> .
Scraper / Extracteur	Inchangés. Toujours nécessaires pour le texte narratif et les publications hors catalogue BDS (ADR 0003).
Interface / Générateur	Inchangés dans leur rôle : mettre en forme et citer la source. La source citée peut désormais être soit un vrai document collecté, soit une fiche synthétique <code>api</code> .

6 Synthèse

Ce complément referme les deux points laissés ouverts par l'ADR 0004 : la traçabilité (résolue par le document synthétique de type `api`) et la fraîcheur des données (résolue par la stratégie cache-aside hybride, figures A et B). Le schéma relationnel et le diagramme de classes originaux restent valides sans modification structurelle : seuls deux colonnes et le comportement de `LookupStructure` évoluent. La prochaine étape est la validation en conditions réelles de `src/bds_client.py`, première tâche du backlog Sprint 2.